Annals of Mathematics and Artificial Intelligence (2025) 93:541–567
https://doi.org/10.1007/s10472-024-09949-3

RESEARCH

Tumato 2.0 - a constraint-based planning approach for safe and robust robot behavior

Jan Vermaelen[1] · Tom Holvoet[1]

Accepted: 14 June 2024 / Published online: 26 July 2024
© The Author(s), under exclusive licence to Springer Nature Switzerland AG 2024

Abstract
Ensuring the safe and effective operation of autonomous systems is a complex undertaking that inherently relies on underlying decision-making processes. To rigorously analyze these processes, formal verification methods, such as model checking, offer a valuable means. However, the non-deterministic nature of realistic environments makes these approaches challenging and often impractical. This work explores the capabilities of a constraint-based planning approach, Tumato, in generating policies that guide the system to predefined goals while adhering to safety constraints. Constraint-based planning approaches are inherently able to provide guarantees of soundness and completeness. Our primary contribution lies in extending Tumato's capabilities to accommodate non-deterministic outcomes of actions, enhancing the robustness of the behavior. Originally designed to accommodate only deterministic outcomes, actions can now be modeled to include alternative outcomes to address contingencies explicitly. The adapted solver generates policies that enable reaching the goals in a safe manner, even when such alternative outcomes of actions occur. Additionally, we introduce a purely declarative manner for specifying safety in Tumato to further enhance its expressiveness as well as to reduce the susceptibility to errors during specification. The incorporation of cost or duration values to actions enables the solver to restore safety in the most preferred manner when necessary. Finally, we highlight the overlap of Tumato's safety-related capabilities with a systems-theoretic approach, STPA (Systems-Theoretic Process Analysis). The aim is to emphasize the ability to avoid unsafe control actions without their explicit identification, contributing to a more comprehensive and holistic understanding of safety.

This paper serves as an invited extension in continuation of our previous work, which was presented at the Third Workshop on Agents and Robots for reliable Engineered Autonomy (AREA) 2023, and builds upon the presented concepts [22]. The current work aims to delve deeper into the constraint-based planning approach, providing further context and clarity to the methodology. We devote more attention to the experiments and elaborate on the evaluation we are conducting, which was previously outlined as future work. Finally, we establish an explicit link with a systems-theoretic approach. For the comprehensiveness of this work, we reiterate the key concepts from our previous contribution.

✉ Jan Vermaelen
jan.vermaelen@kuleuven.be

✉ Tom Holvoet
tom.holvoet@kuleuven.be

1 DistriNet, KU Leuven, 3001 Leuven, Belgium

Keywords Autonomous systems · Behavior specification · Safe and robust planning · Constraint programming · Tumato

Mathematics Subject Classification (2010) 68T40 · 68T42

1 Introduction

Autonomous robotic systems are rapidly gaining popularity in both industry and households. With the expanding scope and complexity of tasks that these systems are expected to handle, the challenge of generating plans ensuring both productive (goal-oriented) and safe behavior becomes increasingly intricate. A plan must be constructed based on the robots' executable actions, environmental information, and desired goals. During this planning process, explicit safety constraints must be considered. These constraints shape the behavior of the system to prevent it from reaching unsafe states. If a safe and productive plan could be generated, it should be **sound** by construction, meaning that the robot will always be instructed with the correct actions.

Our main focus is to support foreseeable non-deterministic transitions while guaranteeing safety. For this purpose, we start from a constraint-based planning approach and build further upon **Tumato**, a state-of-the-art constraint-based planning framework by Hoang Tung Dinh et al. [6]. Tumato operates by translating the specification of a planning problem, including safety rules, into constraints. Solving the obtained constraint problems yields the desired behavior. The approach effectively combines classical planning aspects with a constraint-based backbone to address safety. Further details about Tumato can be found in Section 2.3.

The challenges that we address in this context are twofold. First, moving beyond the assumption of a purely deterministic system, we enable the specification and consideration of foreseeable *alternative effects* of actions. Effects of actions in the real world are virtually never fully deterministic. Non-determinism can arise from varying weights of payloads of robots, various ground surfaces to navigate across, and small measuring and actuation errors, to name a few examples. Second, we enable the explicit specification of *safety conditions*. In the original version of Tumato, achieving a similar result required considering all state-action combinations that could lead to unsafe states. This approach is error-prone and inflexible when the system specification evolves. The generated behavior must always adhere to the safety conditions, even when alternative effects occur.

Since the (physical) environment, and hence the effects of actions, are most often not deterministic in practical robotic applications, the behavior must be sufficiently robust to deal with this kind of uncertainty. For example, the movement of a robot, not being infinitely precise, introduces an inherent uncertainty. Ideally, all contingencies should be taken into account. A first step is to pursue a **complete** policy. The policy should denote the set of actions to be executed for every possible state of the system. As a result, if the system unexpectedly arrives in an unintended state, the operation can continue. We maintain this powerful feature of Tumato's original approach. In a second step, we explicitly account for uncertainty by enabling the modeling of non-deterministic effects of actions, increasing the robustness of the system. In our approach, we assume that each action has one intended outcome, referred to as the *nominal* effects of the action. Additionally, each action can have a number of *alternative* outcomes. These outcomes *could* emerge instead of the nominal one, but they are not intentional. For the goal-oriented aspect of planning, only the nominal outcome is relevant. Alternative effects are unintentional and can not reliably be used to achieve a goal.

However, when considering safety, alternative effects must also be taken into account, as there is a possibility they occur.

Despite these efforts, the system could still arrive in an *unforeseen* state due to external influences. Although safety can not be guaranteed for such events (an external force might put the system in an unsafe state directly), the planner will ensure that the policy contains instructions on how to get back on (safe) track to the goal immediately. If multiple such instructions are possible, the planner is capable of selecting the preferred one based on cost values assigned to actions.

While we enhance the capabilities of Tumato, we seek to position this effort within the broader context of safety analysis methodologies and approaches. Rather than treating this work as an isolated technique, we explore how Tumato can integrate with (software) requirements engineering, with a particular focus on Systems-Theoretic Process Analysis (STPA) [15, 16]. STPA provides a framework to identify, analyze, and mitigate hazards, taking a holistic approach. In the context of autonomous robotic systems, STPA can be used to systematically identify unsafe control actions. The robotic system forming the use case in this paper (see Section 2.2) has received thorough attention during an STPA analysis performed by Buysse et al. [1]. We recognize and examine a partial but significant overlap with the more automated approach embedded within Tumato.

The remainder of the paper is structured as follows. Section 2 outlines the necessary background, including related work, the case study used in this paper, and an overview of Tumato. Section 3 explains and motivates our approach to dealing with uncertainty and safety. Section 4 elaborates on the extension of the specification language, using the robotic system from the case study as an example. Section 5 provides insight into the modified constraints to accommodate our extension and the generated policy. Section 6 provides insight into the results of an initial performance experiment. Section 7 discusses the potential limitations and challenges of the approach alongside ongoing and future work. Finally, Section 8 draws conclusions.

2 Background

Well-thought-out behavior planning is crucial for the safe operation of autonomous safety-critical robots. Consider a scenario where an autonomous robot navigates through a warehouse or factory with potentially harmful tools or payloads in the presence of human agents. The robot's behavior must not only be goal-oriented but also inherently safe. In addition, practical systems often encounter a level of non-determinism that needs thoughtful consideration. Examples are variations in sensor readings, actuation precision, and uncertainties inherent to the environment. While we acknowledge the relevance of all (fully observable) non-deterministic planning approaches, this section primarily serves as a pointer to foundational work, providing the background for the explored planning approach.

2.1 Related work

Traditionally, manual definition has been the standard approach for defining robot behavior. Finite State Machines (FSMs) are often used as a representation [10, 18]. FSMs can model the system's behavior through a structure of states and transitions between them. They provide a comprehensive means to represent a robot's actions based on its state. However, FSMs are known for their limitations in handling the growing complexity of behavior. Missing

transitions or overlooked (error) states are not uncommon, often due to their subtlety. This challenge becomes particularly pronounced when attempting to manually specify sound and complete behavior for larger and more complex systems. In such cases, one has to rely on simulation and verification approaches to ensure that the behavior effectively aligns with the specified requirements. Addressing this challenge involves a shift toward the automatic generation of behavior, making use of a model of the system and a representation of the desired requirements.

Various specification languages and planners have been introduced before. Linear Temporal Logic (LTL) [9] is commonly used. Approaches exist to use (fragments of) LTL to generate FSMs [17, 24] or to compile them into PDDL [2]. The advantage of LTL lies in its ability to account for all contingencies, resulting in sound and complete behavior, a property highly valued in our context. Additionally, explicitly modeling state-based safety conditions in LTL typically requires limited effort. Other examples include Temporal Action Logic (TAL) [7, 8] and the mentioned Planning Domain Definition Language (PDDL) [12, 14], which often rely on replanning at run-time to handle contingencies. However, these approaches do not guarantee the completeness of behavior, as replanning may fail due to an unrealizable specification. This lack of completeness is only detectable at run-time. Finally, in a more practical context, probabilistic planners can be used within the ROSPlan [4] framework directly [3].

To some extent, robustness can be achieved by explicitly addressing non-determinism, as discussed in chapter 5 of the book *Automated Planning and Acting* by Malik Ghallab et al. [13]. By adopting a non-deterministic approach, the planning can leverage the non-deterministic effects of actions to reach the goal. However, in our approach, we deviate from this idea and define a single (intended) nominal outcome for each action while acknowledging alternative (less likely and less desirable) outcomes. This aligns more closely with practical behavior planning problems. It is important to note that in the context of non-determinism, a *safe solution* is defined as a solution in which the goal is reachable from the initial state [13]. This definition differs from the additional safety constraints we impose on the system to achieve the goal *in a safe manner*, starting from any state.

Tomas Geffner et al. present a SAT encoding for fully observable non-deterministic planning [11]. They make a distinction between *fair* and *unfair* non-deterministic actions. In our approach, we focus on fair actions and do not consider adversarial actions or agents. It is important to note that while we recognize the challenges in accurately estimating the probabilities of the effects of actions, we do not want to rely entirely on non-deterministic approaches as we acknowledge the presence of a clear bias in the effects.

In prior research, we conducted an exploratory study of frameworks that combine safe and robust planning [23]. Among the approaches investigated, the use of Markov Decision Processes (MDPs) [20] for probabilistic planning and, to a lesser extent, Simple Temporal Networks (STNs) [5] for temporal scheduling emerged. Their extensions, in particular, are able to explicitly guarantee safety while dealing with uncertainties. Focusing on our current approach, we aim to avoid relying on often inherently imprecise probabilistic values. We explore and extend the constraint-based planning approach Tumato, initially introduced by Hoang Tung Dinh et al. [6]. We elaborate on Tumato in Section 2.3.

Contrasting to the more formal specification and planning-centric approaches discussed above, Systems-Theoretic Process Analysis (STPA) provides a more holistic view on safety [15, 16]. STPA serves as a guiding framework to help identify potential hazards within the system and provide the necessary mitigation. The key steps of STPA include identifying the potential hazards, understanding the system's control structure, determining the unsafe control actions that could lead to those hazards, and analyzing how such unsafe control actions could occur. The analysis also generates the necessary constraints to avoid unsafe control.

By emphasizing the need to identify and mitigate unsafe control actions, STPA contributes to achieving safe behavior.

2.2 Case study

Throughout this paper, we consider a battery-powered Autonomous Mobile Robot (AMR) for validation purposes. We use this system as a case study to identify the shortcomings of the current constraint programming planning approach and to illustrate how to mitigate them. The AMR, shown in Fig. 1, operates within an automated demo factory, as depicted in Fig. 2. The workstations present in the factory can perform various operations on small, standardized workpieces. Some workstations share a conveyor belt, but not all are interconnected. The task of the AMR is to transport workpieces between these workstations. The AMR is equipped with sensing capabilities to detect its state and the necessary actuation capabilities. To maintain focus on the planning approach, we deliberately omit details about the software stack and the computing platform.

The base of the AMR is equipped with three omnidirectional wheels (not visible in the picture), enabling it to move in all directions and turn in place. Given knowledge of the factory layout, the AMR can move to any location and achieve any orientation within the factory. The AMR can navigate between workstations, a charging station, and the connecting *corridor*. Its sensory capabilities include a camera and LIDAR system, complemented by a number of proximity sensors around the base. On top, a fixed-height looping conveyor belt system is mounted, capable of holding one workpiece. With this equipment, the AMR can dock to workstations and receive and deliver workpieces. Infrared sensors attached near the conveyors can further aid with the alignment when docking. Finally, the AMR is equipped with an emergency stop button as well as a beacon for visual signaling.

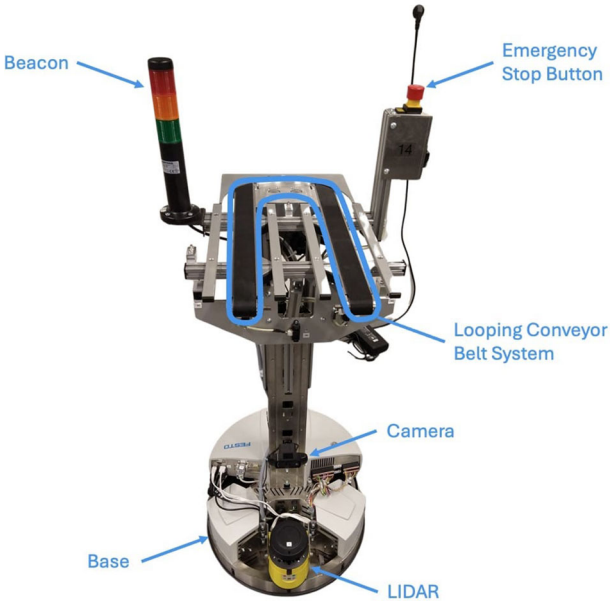

Fig. 1 The AMR with a LIDAR system and camera mounted at the base and conveyor system, emergency stop button and light beacon present on top

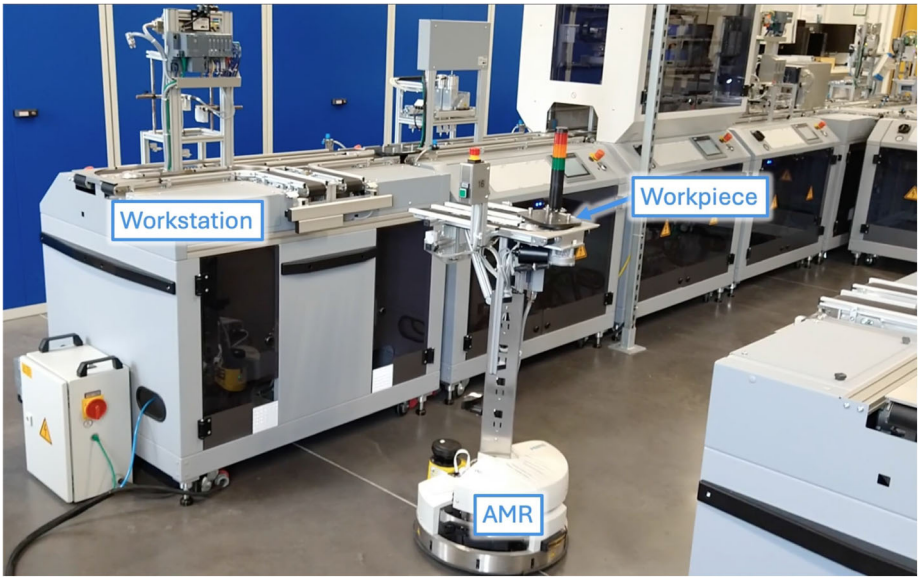

Fig. 2 Part of the demo factory with an AMR (carrying a workpiece) located in front of a workstation

One or more AMRs can operate simultaneously on the factory floor alongside human agents. The absence of a physical separation between AMRs and humans increases the significance of generating safe behavior. Adding to the challenge, the effects of actions unintentionally vary, even within a relatively controlled environment like a factory. An AMR should complete its assigned transport task without giving rise to any unsafe situations, even under these circumstances.

2.3 Tumato

Tumato is a planning approach, achieving sound and complete behavior through constraint programming. The specification of a system includes information about the states of the system, the environment, the executable actions, and the goals, as well as a set of safety rules. Tumato effectively combines classical planning aspects with constraint programming to enforce safe behavior. In this section, we provide a comprehensive explanation of the approach. For further details, please refer to the original work on Tumato [6].

Constraint-based planning in Tumato involves the automated process of converting the entire system specification into constraints. To initiate the planning process, one Constraint Satisfaction Problem (CSP) [21] is generated for each valid initial state, with a given maximum planning length. The initial state forms the first constraint. Next, for each step in the plan, constraints are added. Constraints are introduced to address mutually exclusive actions arising from resource conflicts. Preconditions of actions are translated into constraints to determine their executability, and constraints are formulated to represent the effects of actions. The safety rules, which inherently resemble constraints, are integrated as well. The final steps involve including assumptions and goals. Whenever conflicts arise among plans generated for different initial states, Tumato dynamically adds constraints until a coherent policy is obtained.

The set of CSPs, derived from the specification, is solved offline before the system is deployed. Currently, Tumato employs Choco-solver [19], although the approach generally

aims to remain solver-agnostic. As each CSP corresponds to the plan for one initial state, the set of actions executed in the first step of the plan is selected for that state. The final result is a mapping from every state to the actions that have to be executed in that state, which we refer to as the policy. If a solution exists, we consider the specification realizable. Otherwise, we say the specification is unrealizable. If the specification is realizable, Tumato will generate a sound and complete policy, which can be used at run-time without requiring online re-planning. In case of an unrealizable specification, where for at least one of the initial states, no (conflict-free) plan can be found, this information will be provided as feedback to the user. Examples include states for which the system can not guarantee safety and states for which the goals can not be satisfied within the given maximum planning length.

In the remainder of this section, we provide an overview of the specification in Tumato, using a model of the AMR as an example. Excerpts from the specification are included. Our work will extend this specification where necessary, see Section 4.

States

State vectors are used to represent the states in which the system can reside. A state vector consists of a set of discrete state variables. Each of the state variables represents one aspect of the state. At run-time, the current state (vector) is to be determined by a monitoring module. This module is responsible for translating the sensor readings and other input to discrete values for the state variables. An example of a state variable is the location of the AMR. The state variable $S_{Location}$ can represent the different discrete locations at which the robot can reside, including a common corridor, a charging station, and two interactable workstations. Additional workstations and locations can be added as needed. $S_{Location}$ is treated as a topological representation. While this representation results in a straightforward specification, the lower-level movement module will handle the actual navigation. Alternatively, if more detailed location information is required, a more fine-grained location discretization can be used. This approach simplifies the lower-level modules and allows for more explicit movement planning. We specify the following state variables for the AMR:

```
BEGIN STATE VECTOR
    state S_Location can be corridor, charger, workstation_1, workstation_2
    state S_Battery can be low, ok
    state S_Load can be loaded, free
    state S_Conveyor can be on, off
END STATE VECTOR
```

Please note that the use of the prefix ”S_” for state variables is merely a modeling choice.

Resources

Different resources can be specified to be used by the actions. The purpose of allocating resources is to prevent actions from being executed simultaneously if they make use of at least one resource in common. For example, two different *move*-actions requiring the resource *motors* can not execute at the same time. The specification of the AMR contains two resources:

```
BEGIN RESOURCES
    resource MOTORS
    resource CONVEYOR
END RESOURCES
```

Aliases

Acting mainly as syntactic sugar, aliases can be introduced to give a dedicated name to predicates. This is useful to avoid cluttering the specification. We can define an alias for the AMR being docked:

```
BEGIN ALIASES
    define is_docked as S_Location is workstation_1 OR S_Location is workstation_2
END ALIASES
```

Assumptions

Assumptions can be added to specify knowledge about the system and its environment. The planner will only consider states in which the assumptions are satisfied. As an arbitrary example, consider a state variable $S_{Charging}$ to detect whether or not the robot is currently charging. $S_{Charging}$ can only obtain the value *active* when the robot is located at the charger:

$$S_{Charging} = active \Rightarrow S_{Location} = charger.$$

States not satisfying the assumptions are considered invalid and are assumed never to occur, as they are excluded from the policy. Adding assumptions where possible can greatly improve the performance of the planner. For the current AMR example, we do not specify any assumptions.

Actions

Actions represent elemental behavior, expressing the smallest units of execution to be considered during planning. For instance, both *move_one_cell_forward* and *move_to_charging_station* are viable actions, yet operate at different abstraction levels. Actions can be defined with specific *preconditions* that must be satisfied before execution. Actions may also involve the use of *resources*, as introduced earlier. Finally, actions typically induce an *effect* on the system's state. As an example, consider the action **move_to_workstation_1**:

```
// Please note, comments start with "//".
BEGIN ACTIONS
    action move_to_workstation_1
    controlled resources: MOTORS
    preconditions: S_Location is corridor
    effects: S_Location is workstation_1

    // ... other actions
END ACTIONS
```

Reaction rules

Reaction rules, as the name suggests, can be used to specify reactive behavior. They consist of logical rules based on the current state and executed actions. The rules indicate that if a certain condition holds in the current state, a specific action must be executed or is prohibited from execution. For example, consider the scenario where the AMR is in the corridor, and the workstation-sensitive conveyor belt is active. In this case, the conveyor should be deactivated, achieved by executing the action **stop_conveyor**. The corresponding reaction rule is specified:

```
BEGIN REACTION RULES
    rule: IF S_Location is corridor AND S_Conveyor is on THEN executing stop_conveyor

    // ... other reaction rules
END REACTION RULES
```

For a more in-depth exploration of the application and implications of reaction rules, please refer to Section 4.2.1.

Goals

Goals define the conditions that the system must aim to achieve. Given the constraint-based approach, a variety of goal formats can be specified. We focus on *prioritized* and *conditional* goals. A conditional goal is active only when a specified condition is met. *If the AMR is loaded with an item, the goal is to unload that item* (2), that is, deliver it. Similarly, a conditional goal is specified for picking up an item (3). Priorities are used to determine which conditional goals should take precedence. A third conditional goal is defined to recharge the battery when it reaches a *low* level (1). This goal is prioritized over the transport goals (2) and (3). Priorities are indicated based on the order in which the goals are specified:

```
BEGIN GOALS
    goal type: priority
    when S_Battery is low then goal: S_Battery is ok // (1)
    when S_Load is loaded then goal: S_Load is free // (2)
    when S_Load is free then goal: S_Load is loaded // (3)
END GOALS
```

It is important to note that, for a practical application, the monitoring module should update $S_Battery$ to ok only when the battery has been sufficiently charged. An earlier update would result in undesired shorter operation cycles.

Configuration

The specification concludes with a *config* block, currently containing only the maximum planning length:

```
BEGIN CONFIG
    max_plan_length: 5
END CONFIG
```

This parameter indicates the maximum number of steps considered within each Constraint Satisfaction Problem. The Tumato solver employs an incremental approach, looking for the shortest solution for each planning problem up to this value.

3 Dealing with uncertainty and safety

As mentioned in Section 1, we enhance robustness by incorporating foreseeable alternative outcomes for actions. This section employs a grid-based navigation planning example to introduce the extensions and their associated methodology and motivation. A more comprehensive understanding can be gained from the case study example, see Section 4.

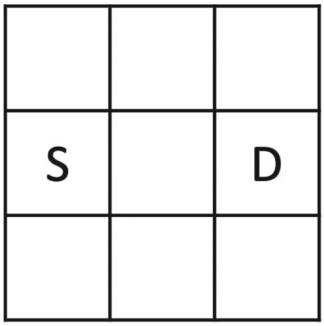

Fig. 3 Illustrative 3 × 3 grid example with starting point *S* and destination *D* [22]

3.1 Uncertainty

Consider a 3×3 grid, with a starting point *S* and destination *D* as depicted in Fig. 3, to define the navigation planning problem. Each *cell* of the grid represents one state. We introduce the actions *move_up, move_down, move_left*, and *move_right* to *move* between states. In a deterministic setting, each of these actions reliably moves the agent one cell in the intended direction. In the example, the shortest plan from starting point *S* to destination *D* involves executing *move_right* twice. A complete policy for this planning problem is illustrated in Fig. 4.

In a more realistic, non-deterministic planning setting, the success of actions is not guaranteed. Firstly, an action may simply have no effect. For example, the agent might not have moved a sufficient distance to reach the next cell. Since the behavior is defined as a policy, the same action will be executed again, presumably leading to a new state eventually. It is important to recognize this implicit assumption when constructing the system. If an action can fail indefinitely, such as when the agent lacks the necessary force to move up a hill, the action has to be reconsidered. Either the model is not adequate (the specified effect does not align with the real action), or conversely, the (physical) implementation of the action has to be adapted.

Secondly, deviations may occur, leading to unintended alternative outcomes. When the *move_right* action is executed from the starting point *S*, two states (or cells) could be reached unintentionally, as indicated in Fig. 5. These outcomes may be identified after deploying and observing the system for an extended period or may be specified by domain experts. While these alternative outcomes are less likely than the desired and intentional (nominal) outcome, their potential occurrence requires consideration, especially in the context of safety.

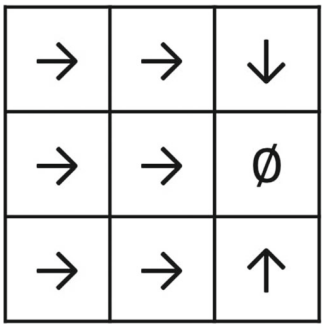

Fig. 4 Policy to reach destination *D* assuming a deterministic approach [22]

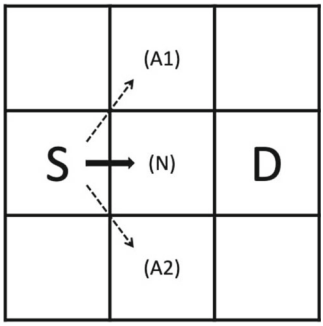

Fig. 5 *Move_right* in S with nominal outcome (N) and two alternative outcomes [22]

Since they are assumed to occur only sporadically, accurate probabilistic values can not be determined and used during planning.[1] For the productive, goal-oriented aspect of planning, we do not have to consider alternative outcomes, and the planner may assume that the intended outcomes occur. In case the system was to arrive at any unintentional state (possibly due to an alternative outcome), the completeness of the policy ensures that the operation can continue to reach the goal. If from any state the goal can not be reached, no policy can be found, and the planner will provide such states as feedback to the user.

Note on non-determinism

The presented concept of differentiating between nominal and alternative outcomes can also be used to model *true* non-determinism in the system. As an example, consider a (potentially unfair) coin being flipped at run-time to decide the outcome of an action. In the specification, two actions, one corresponding to each side of the coin, have to be modeled to eliminate the inherent bias towards the nominal outcome. For each action, the nominal outcome accounts for one side of the coin, while the alternative outcome accounts for the other side. Regardless of the outcome of an individual toss, the policy ensures that the executed actions are safe, as we define next. This approach can be extended to an n-sided die where n actions with n-1 alternative outcomes have to be modeled.

3.2 Safety

We expand upon the previous grid example, now considering a 5×5 grid. In this setup, the outer cells are marked as unsafe. These cells might be located next to stairs, which the agent could potentially fall off, or a wall it could collide with. While this concept of simply identifying states or conditions as unsafe is rather straightforward, it results in a powerful specification. We experience a large improvement over manually identifying unsafe state-action combinations, as is required using reaction rules.

Executing a *move_right* action at the starting point S will never lead to an unsafe state, as shown in Fig. 6. This action is allowed to be selected during planning. In turn, in both of the unintentionally reachable states (labeled as *A1* and *A2* in the figure), a similar *move_right* is not allowed, as the alternative outcomes could lead the agent into an unsafe state. A safe policy is depicted in Fig. 7. Note that many cells remain empty, as those cells do not allow

[1] If precise probabilistic values for outcomes can be determined, a Markov Decision Process (MDP) would be a more suitable approach for obtaining the policy.

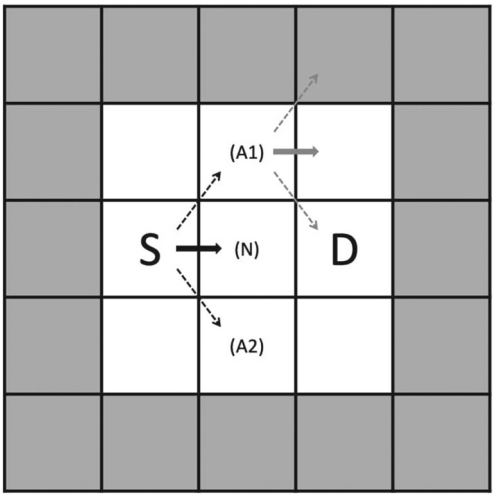

Fig. 6 Three states (N, $A1$ and $A2$) are reachable by executing *move_right* in S. A second *move_right* in either $A1$ (visualized) or $A2$ (not visualized) could lead to unsafe (gray) states. A *move_right* in N will always be safe [22]

any safe actions in our current example. To obtain this policy, the specification would need to explicitly include a number of assumptions to exclude those cells from the valid state space. If those assumptions are not added, no result can be obtained since no complete policy exists.

Note on restoring safety

The introduced approach does not differentiate between starting from a safe or unsafe state. It merely guarantees that the next state will be safe. For unsafe states, a solution will only be found when safety can immediately be restored. Moreover, the planner is aware that when multiple actions are executed simultaneously, the effects of only *one* of those actions will take place *first*. Therefore, every possible outcome of each action selected for execution must be safe. Furthermore, as actions can be assigned a cost value (see Section 4.1), the planner

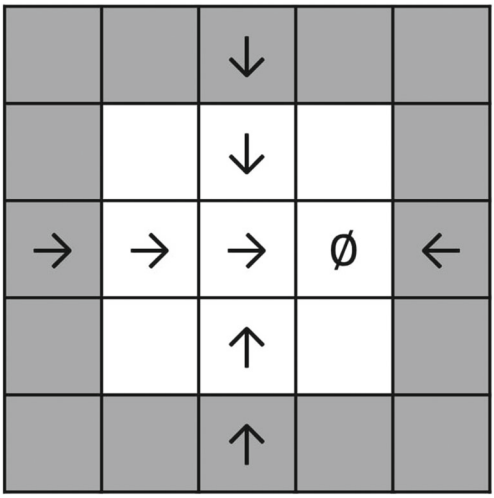

Fig. 7 A guaranteed safe policy. This policy can **not** be considered complete and should be accompanied by the necessary assumptions stating that the *empty* cells can never occur. Such assumptions might not be realistic [22]

is able to choose the set of actions that restores safety in the most preferred way, according to these costs.

4 Extending the specification

In this section, we elaborate on the modifications made to the specification language, outlined in Section 2.3. The primary contributions involve the incorporation of alternative effects and costs of actions, as well as the introduction of state rules. To illustrate these additions, we continue to use the AMR detailed in Section 2.2. For the complete specification conforming to Tumato's updated format, please refer to Appendix A.

4.1 Actions

In contrast to the deterministic approach of the original Tumato framework, we differentiate two types of effects. *Nominal effects* represent the intended and desired outcome of an action, while *alternative effects* denote less likely unintended outcomes that *could* manifest instead of the nominal one. To ensure a comprehensive understanding of both types of effects, we present a dual perspective. In the context of productive (goal-oriented) planning, only the nominal effects are taken into consideration, as detailed in Section 3.1, deliberately excluding undesired outcomes. However, in the pursuit of ensuring safe behavior, *all* effects must be taken into account, see Section 3.2. Every potential outcome resulting from the execution of an action must be safe. This two-fold perspective facilitates an appreciation of both kinds of effects within their respective formal contexts.

We further extend the specification of actions with a generic cost. In the AMR example, the notion of a *relative duration* is used. If no duration is provided, the planner will assume a value of *0*. The planner only considers these values when finding the best solution to restore safety. For states where safety should only be maintained rather than restored, these values are ignored. Please note that *time* is not taken into account explicitly. The durations are used solely to find the most preferred (fastest) way to restore safety.

Next, we present a few examples of actions of the AMR for further clarification.

```
BEGIN ACTIONS
    action stop_conveyor
    duration: 1
    controlled resources: CONVEYOR
    preconditions: S_Conveyor is on
    nominal effects: S_Conveyor is off
```

Given that halting the conveyor happens virtually instantaneously, the action is assigned an arbitrarily low duration value. Since no alternative effects are specified, it is assumed that the action will never fail.

```
    action move_to_workstation_1
    duration: 10
    controlled resources: MOTORS
    preconditions: S_Location is corridor
    nominal effects: S_Location is workstation_1
    alternative effects: S_Location is corridor
```

Moving to a designated location gets assigned a relative duration of *10*. The nominal outcome is, as expected, successfully reaching the specific workstation. The alternative outcome entails the AMR failing to reach the location. In practice, this alternative effect may occur if

the specified workstation is obstructed or already occupied. Please note that this possibility is specific to the workstation in question, and the effects can be tailored differently for each action (see move actions in Appendix A).

```
    action receive_workpiece
    duration: 3
    controlled resources: MOTORS, CONVEYOR
    preconditions: S_Location is workstation_1, S_Load is free
    nominal effects: S_Conveyor is on, S_Load is loaded
    alternative effects: S_Conveyor is on, S_Load is free
    alternative effects: S_Conveyor is off, S_Load is free

    // ... other actions
END ACTIONS
```

Workpiece transfer actions are allocated a relative duration of *3*. During the transfer process, the motors are used to stabilize the AMR. In the current example, acquiring workpieces is limited to *workstation_1*, as indicated by the precondition. Two sets of alternative effects are specified for this action. The first set represents a scenario where a workpiece, perhaps due to an irregular shape, becomes lodged. The second set represents a more severe outcome where the entire conveyor is obstructed by receiving a workpiece that is, for instance, excessively heavy.

These three examples illustrate the modeling of actions and their effects. When only nominal effects are provided, the outcome of the action is considered deterministic. Alternatively, the specification allows for the inclusion of one or more alternative outcomes. While we aim for concise specifications, it is necessary to include all foreseeable alternative effects to ensure safety. The (potentially combinatorial) overhead caused by actions having multiple effects remains limited, as alternative effects are only evaluated on a per-state basis to apply the state rules. In practice, the number of relevant actions (based on preconditions and goals), combined with the number of effects, remains relatively small. The larger concern remains in the combinatorial explosion in longer paths of states and actions during the planning process. Finally, please recall that these alternative outcomes are not considered in the productive aspect of planning. We do not (want to) rely on the alternative effects to reach a particular goal. They are only considered when guaranteeing safety.

4.2 Safety rules

Safety rules are used to impose constraints on the behavior of the system to guarantee safe behavior. In this section, we first delve deeper into the use of the existing reaction rules and then introduce the new state rules.

4.2.1 Reaction rules

As mentioned in Section 2.3, reaction rules specify reactive behavior. When the specified condition holds in the current state, an action is either forced to be executed or prohibited from execution. We already specified the example where, if the AMR is located in the corridor with the conveyor on, the conveyor should be stopped:

$$(S_{Location} = corridor \wedge S_{Conveyor} = on) \Rightarrow Exec(\textbf{stop_conveyor}).$$

We expand this rule to also include the charger location:

$$(S_{Location} = corridor \vee S_{Location} = charger) \wedge (S_{Conveyor} = on) \Rightarrow Exec(\textbf{stop_conveyor}).$$

However, we argue that the conveyor should never be on in the first place when the AMR is either in the corridor or at the charger. Such behavior could lead to dropping workpieces and other hazardous situations for human agents in the factory. Given our knowledge of the system, we can express:

$$(S_{Location} = corridor \vee S_{Location} = charger) \wedge (S_{Conveyor} = off) \Rightarrow \neg Exec(\textbf{receive_workpiece}) \wedge \neg Exec(\textbf{deliver_workpiece}).$$

This rule needs to be updated for every new action that *could* activate the conveyor. Additionally, we must include rules like:

$$(S_{Conveyor} = on) \Rightarrow \neg Exec(\textbf{move_to_\{loc\}})$$

for every location *loc* where moving toward *loc* could lead to reaching the corridor or charger. This last expression can give rise to a large number of reaction rules, depending on the number of locations and how the effects of different move-actions connect them. In Tumato, we specify:

```
BEGIN REACTION RULES
    rule: IF (S_Location is corridor OR S_Location is charger) AND S_Conveyor is on
        THEN executing stop_conveyor
    rule: IF (S_Location is corridor OR S_Location is charger) AND S_Conveyor is off
        THEN NOT executing receive_workpiece AND NOT executing deliver_workpiece
        // AND NOT any future action that could turn the conveyor on
    rule: IF S_Conveyor is on THEN NOT executing move_to_corridor AND NOT executing
        move_to_workstation_1 // AND NOT any future action that could move the AMR
        //away from NOT(corridor OR charger)
    // And probably more rules
END REACTION RULES
```

Please note that a lot of reaction rules will have to be reconsidered and updated to accommodate changes to the specification in the future.

4.2.2 State rules

To significantly reduce the number of required safety rules and to make the specification less prone to errors, we introduce *state rules*. Unlike reaction rules, which impose constraints on actions, state rules enable the specification of constraints on reachable states. In line with reaction rules, state rules also adhere to propositional logic and maintain the *IF-THEN* format. However, in this case, state rules constrain (next) states rather than (current) actions. We illustrate state rules starting from the previous example, where multiple reaction rules were necessary. Now, with a single state rule, we can specify the constraint more succinctly:

```
BEGIN STATE RULES
    rule: IF S_Location is corridor OR S_Location is charger THEN S_Conveyor is off
END STATE RULES
```

We present state rules in the form of *desired safety conditions*. These conditions should always remain *True* to the best of the system's abilities. Alternatively, one could specify unsafe conditions, which should be kept *False*. Translating between the two corresponds to negating the conditions.

The effects of any executed action should never lead to the violation of a state rule. Formally, a state rule expresses a condition that must hold after executing any (set of) action(s) instructed by the policy, irrespective of the order in which actions effectuate and regardless of which (nominal or alternative) effects occur. Whether an action a is permissible in a state s can be expressed as follows:

$$allowed(a, s) \Leftrightarrow \forall effect \in effect(a, s) : effect \Rightarrow \{state_rules\}$$

where $effect(a, s)$ contains all possible effects ($nominal \cup alternative$) of action a in state s and $effect \Rightarrow \{state_rules\}$ denotes that the effect adheres to all conditions described by the state rules.

If the current state were to violate a state rule, the next planned (set of) action(s) will always resolve that violation (see Section 3.2). Given that the state rules prohibit any actions that could knowingly lead to undesired states, the system must have reached that state under the influence of external forces. During planning, if no instructions exist to restore safety in a particular state, the planner notifies the user, specifying which state lacks a feasible solution. In scenarios where multiple such instructions are available, the planner selects the set of instructions that restores safety in the most preferred way. In this example, durations are assigned to actions, indicating the aim of restoring safety as quickly as possible. Alternative approaches include utilizing costs to identify the best solution or explicitly addressing risks.

In some applications, restoring safety might not be feasible. It is possible to add assumptions to the model, excluding the unsafe states described by the state rule(s). The state rules will still make sure that the unsafe states are not knowingly reached, yet for those states, the planner will provide no instructions. As an illustrative example, it is highly undesirable for a robot to completely drain its battery. Such states can be indicated using state rules and are to be avoided. However, if at some point, the robot obtains a completely drained battery, no actions are feasible.

4.2.3 Relation between reaction rules and state rules

While a formal analysis of the overlap between the two types of safety rules is not included in this work, their unique capabilities and practical relevance are discussed.

Firstly, reaction rules can express constraints that state rules cannot cover. For instance, they can selectively address actions leading to unsafe states rather than considering all possible ways of reaching those states. Additionally, reaction rules can handle actions that have no direct effect on the state.

Secondly, while state rules used for maintaining safety can often be replaced by a more elaborate set of reaction rules, state rules are also powerful for restoring safety. Although reaction rules can also try to restore safety to some extent, the functionality of restoring safety in the most preferred way can not be achieved by only using reaction rules. This would require reaction rules to be specified to the point of individual states, making the planning a fully manual effort.

The core difference between reaction rules and state rules can be summarized as follows: reaction rules focus on reactive behavior, consisting of the executed actions based on the current state, whereas state rules focus on the potential outcomes of the executed actions. Overall, state rules are more strict, while reaction rules are more fine-grained and require more input from the user. Both approaches have practical applications that cannot be fully covered by the other. However, for the general purpose of avoiding unsafe states, both can be used effectively, with state rules offering advantages such as fewer rules and easier expression.

5 Extending the planner

In this section, we first summarize the modifications made to the planner to accommodate our extensions. Next, we elaborate on the generated policy using the example specification of the AMR.

5.1 Constraint-based planning

The constraint-based approach providing the basis for this work has been introduced in the original work on Tumato [6]. By implementing our extensions (see Section 4), we introduce two new main sources of constraints, as well as an optimization objective.

First, alternative effects of actions have to be translated into constraints, similar to the original (or nominal) effects. Although this is quite a straightforward addition, it gives rise to *a set of* next possible states per action, rather than just one.

Second, constraints related to the new state rules are introduced. While reaction rules could be translated directly into the constraint satisfaction problem as constraints that prohibit or enforce actions to be executed, state rules require more insight. For each execution step in the plan, the state rule is applied as a constraint to each state in the set of *reachable next states*. In other words, state rules introduce constraints on all possible outcomes of an action rather than on executing the actions. This results in a number of new constraints, one for each possible outcome, enforcing the state rule conditionally to whether the action gets executed. As a result, if an action *could* violate one or more state rules, this action will be prohibited in the given state. Practically, these constraints replace a (potentially large) number of constraints individually specified by reaction rules, as illustrated in Section 4.2.1. Complementary, state rules can also enforce the execution of actions if their effects are required to maintain safety. Even if the current state is not safe, the next state is guaranteed to be safe, and the planner will now minimize the required duration or cost to restore safety using a minimization objective. Since the use of objectives is solver-specific, we refrain from elaborating further and only illustrate their use. Finally, when no solution exists that satisfies all constraints, the solver can refer to states for which no behavior can be generated to *explain* why the specification is unrealizable, identical to the original feature of Tumato.

5.2 The policy

The generated behavior is represented as a policy, containing a mapping from states to (sets of) actions. Since the generated policies are complete, every valid state of the system has a corresponding entry. States that have been excluded, using assumptions, are considered invalid. In this section, we present a few well-chosen entries from the AMR's policy:

```
"S_Location": "corridor",
"S_Battery": "ok",
"S_Load": "free",
"S_Conveyor": "off",
"Actions": ["move_to_workstation_1"]
```

This first entry instructs the AMR to move to workstation_1, given that the AMR is not carrying a workpiece and sufficient battery power remains. The instruction aligns with the desired productive behavior.

```
"S_Location": "corridor",
"S_Battery": "low",
"S_Load": "free",
"S_Conveyor": "off",
"Actions": ["move_to_charger"]
```

In the second entry, a different (prioritized) conditional goal is active due to the battery level holding the value *low*. The AMR now has to move to the charger.

In both the first and the second entry, the state rule condition (specified in Section 4.2.2) was satisfied, with no violations in the current state nor in any foreseeable possible next state.

```
"S_Location": "workstation_2",
"S_Battery": "ok",
"S_Load": "free",
"S_Conveyor": "on",
"Actions": ["stop_conveyor"]
```

The third entry shows the influence of the specified state rule. In the absence of this rule, the planner would typically instruct the AMR to start moving toward *workstation_1*. Given the later use of the conveyor, it would be redundant to instruct the AMR to turn off the conveyor at this point. The state rule guides the system to execute the *stop_conveyor* action instead. It is worth noting that an ill-considered implementation of state rules (or the use of reaction rules) might allow the simultaneous execution of both movement and stopping the conveyor. Yet, the adapted Tumato solver recognizes the non-deterministic order in which actions effectuate when multiple actions are executed simultaneously and ensures safety by permitting only the *stop_conveyor* action.

```
"S_Location": "corridor",
"S_Battery": "ok",
"S_Load": "loaded",
"S_Conveyor": "on",
"Actions": ["stop_conveyor"]
```

In the final presented entry, the current state is violating the state rule. To restore safety, the AMR is instructed to execute *stop_conveyor*. Although the more productive action *move_to_workstation_2* would also restore safety (as the only foreseeable next state is safe), the planner chooses the most preferred way to restore safety based on the durations of actions.

The complete policy can be found in Appendix B. A short clip of the AMR executing a policy generated by Tumato can be found here: *https://youtu.be/UFs4DljBRKk*.

6 Performance evaluation

In this section, we present the findings of our experiments, aiming to provide insight into the performance of Tumato. We recognize that performance encompasses various perspectives and aspects. Our focus includes (1) the policy generation process, which is the subject of the results presented in this section, and (2) the time and effort required by a user to model a system correctly, which is currently under evaluation in ongoing work.

We compare the use of reaction rules with the use of state rules in terms of the number of generated constraints and the execution time required to generate the policy. The specification indicated as *RR* only makes use of reaction rules. Analogously, *SR* only makes use of state rules. To the best of our knowledge, the defined reaction rules enforce the same safety guarantees as the state rule, and we have validated the resulting policies to be identical. For both specifications, a maximum planning length (MPL) of both 5 and 10 is evaluated. For

Table 1 Specifications of the experiment models, including the number of states and actions and the presence (and number) of reaction rules, state rules, and assumptions

Specification	#States	#Actions	React rules (#)	State rules (#)	Assump. (#)
RR	32	8	✓ (3)	× (0)	× (0)
SR	32	8	× (0)	✓ (1)	× (0)
Large	4608	18	✓ (6)	✓ (1)	✓ (1)

more details on the specification, see Section 4. The complete specification can be found in Appendix A. The resulting policy can be found in Appendix B. While the experiments mainly focus on the described model of the AMR, we also include the results for one larger specification, *Large*, for a scalability comparison. A comprehensive overview of the specifications used in the evaluation can be found in Table 1.

Please note that the number of state rules and reaction rules in the overview only serve indicative purposes. Multiple rules could be combined into one more complex rule or vice versa. Overall, we observe that reaction rules require more thought and can be considered more complex.

The observed amount of generated constraints and internal variables are indicated as a range in Table 2. The first value indicates the initial amount, which is constant across all starting states. For some of the starting states, the number ramps up as constraints are added automatically to obtain a conflict-free policy. The second value is the highest observed value across all starting states.

Increasing the MPL does not have an influence on the constraint satisfaction problems, as can be observed by comparing *RR (5)* with *RR (10)* and comparing *SR (5)* with *SR (10)* in Table 2. We find confirmation that when a solution can be found, a larger MPL does not affect the planning problem as Tumato incrementally looks for a solution. Although we omit the experimental results, it is worth noting that experiments conducted with the same model confirm that introducing and making use of one or more aliases does not impact the policy generation metrics or the resulting policy.

The larger specification, *Large*, uses both reaction rules and state rules and includes one assumption. The assumption reduces the number of valid states (see *Large* in Table 2) and hence reduces the required computational effort. Recall that the policy will not contain any actions for invalid states.

Table 2 Empirical results of the experiments for the given MPL, including the number of valid states, constraints, and variables generated by the solver

Specification (MPL)	#Valid states	#Constraints	#Variables	Time (s)
RR (5)	32	434 – 2870	692 – 2998	0.93
RR (10)	32	434 – 2870	692 – 2998	0.84
SR (5)	32	551 – 3338	877 – 3553	1.23
SR (10)	32	551 – 3338	877 – 3553	1.26
Large (10)	2688	1190 – 5725	1886 – 8806	74.29
Large (20)	2688	1190 – 5725	1886 – 8806	76.48

The time required to find a solution has been averaged across 10 measurements on a *1.6 GHz Dual-Core Intel Core i5*

Experiments were also conducted for models where no solution (policy) can be found. While the same number of constraints and variables are applicable, the generation time (until failure) greatly varies based on when, during the planning, the critical conflict is detected. For such models, we do see the MPL affect the metrics as a larger MPL allows the planner to try and look for longer plans to find a solution. For the specifications *RR* and *SR*, a solution can be found using an MPL of *4* or higher. We refrain from including further experiments and measurements for varying numbers of states, actions, and rules, as the computational effort is more closely linked to the complexity of the model rather than its mere size. Finally, no significant memory usage was detected in any of the experiments.

We see that using the state rule approach, the numbers of constraints and variables start slightly higher compared to the reaction rule approach yet remain within the same order of magnitude. We refer to the comparison between *RR* and *SR* in Table 2. Under the hood, state rules generate constraints that effectively replace the need for individually defined reaction rules. Since reaction rules necessitate manual implementation, which is more error-prone, state rules might cover situations the user did not anticipate. This is especially beneficial for more complex systems.

Finally, we have observed only a small influence of *restoring safety the most preferred way* on performance, even when the number of unsafe states grows. For those states, the *best* solution has to be found, while otherwise, *any* solution is sufficient. While a complete overview and analysis of these observations are beyond the scope of this work, we believe it is valuable to share these initial insights.

For readers referencing the results in [22], the specification *SR* with an MPL of *5* aligns with our previous experiment. The slight increase in the starting number of constraints and variables compared to previous work is attributed to minor improvements in our approach's implementation.

7 Discussion

In this section, we discuss the challenges and potential limitations of the approach and provide insights into ongoing and future work.

Tumato successfully connects the theoretical agent-based research, particularly on guaranteeing safe behavior, with the field of robotics, where non-determinism is inevitable. The AMRs in the demo factory successfully execute policies generated by Tumato. By including durations and alternative outcomes, we achieve a concise and expressive behavior specification. Additionally, the introduction of state rules further enhances expressiveness and results in a less error-prone safety specification. However, a notable limitation of Tumato lies in the need for discrete state variables. The practical application of the approach requires the inclusion of a monitoring module to map the continuous world to a discrete state space.

Tumato and STPA

In light of ensuring safe control, it is valuable to explore the potential integration between the Tumato framework and a Systems-Theoretic Process Analysis (STPA) approach. Both methodologies can contribute to the safety of the system's behavior. This integration focuses on the identification and mitigation of unsafe control actions. An STPA analysis traditionally involves four key steps: i) understand (the purpose of) the system and identify potential losses and hazards; ii) construct the control structure, including controllers, (human) operators, and other processes; iii) identify unsafe control actions by examining potential outcomes of

control actions in different system states; and iv) examine scenarios that could lead to the occurrence of unsafe control actions. While the first two steps serve as a general starting point to understand the system, the third and fourth steps represent the main STPA approach. The third step examines if the execution or omission of a control action is unsafe. The output of this step can be interpreted as constraints on when actions are not allowed to be executed or must be executed. These constraints closely relate to Tumato's concept of reaction rules and can be incorporated directly into the specification. In the fourth step, STPA investigates why and how unsafe control actions might occur. Usually, the process models and control algorithms are investigated. To automatically obtain such guaranteed safe control, the new state rules of Tumato can mitigate and avoid unsafe control by design. While the responsibility for the correctness of the model still lies with the designer, Tumato's specification can facilitate this task. This integrated perspective highlights Tumato's shift towards automated safety mechanisms, leveraging state rules to address all possible action outcomes. As part of ongoing work, we are actively investigating more detailed comparisons and integration between STPA and Tumato.

Participant-based empirical evaluation

We are conducting a participant-based empirical evaluation to assess the practical use of the adapted planner. The goal is not to compare the approach with state-of-the-art planners for several reasons. Given that policies are generated offline, real-time performance is not a concern. Furthermore, the soundness and completeness of the approach ensure safety and productivity, meeting our needs. Instead, the study aims to evaluate the actual (safety) guarantees that can be obtained by the average user, along with the perceived specification effectiveness and comfort. Please note that the average user wanting to deploy a robot is typically not a robot expert. We are particularly interested in comparisons with more traditional and commonly practiced ad-hoc approaches, which should require a similar effort from the user. A comparison with the original version of Tumato is in order as well. As a starting point, the empirical evaluation aims to obtain valuable insights into the behavior planning landscape for autonomous robots. To analyze various approaches, we have invited a number of participants to implement a simple robot comparable to the AMR described in this work. The necessary context, environment, and a high-level API for the robot are provided, and the behavior can be implemented following any approach of the participant's choice. After the participants have implemented the behavior, we evaluate their approach and examine in what facets Tumato would do better or worse. By focusing on the participants' experiences and considering factors such as (time) efficiency, effort, and usability, the evaluation seeks to provide a holistic understanding of different behavior planning approaches. The results of this study are expected to enhance our insight into typical ad-hoc behavior planning approaches and allow us to evaluate Tumato accordingly.

Future work

Despite the promising combination of uncertainty and safety in the constraint-based planning approach, there are still challenges that need further exploration. The state rules ensure that every foreseeable outcome of executed actions adheres to the safety requirements. The planner, capable of selecting the optimal set of actions for restoring safety, notifies the user if, for some states, immediate safety restoration actions are unavailable. While this approach is valuable, future research could explore the idea of allowing multiple successive sets of

actions in unsafe states if necessary to restore safety. Additionally, in practical scenarios, safety constraints often vary in severity. Similar to assigning specific values to actions, the weighting of safety rules could be considered. This addition would enable the constraint solver to identify an overall optimal solution, accounting for the varying degrees of severity associated with different safety constraints. Finally, the use of, or link with, Linear Temporal Logic to specify the safety rules should be further investigated as well.

8 Conclusion

This paper presents an approach to specify and generate safe and robust robot behavior. The extension of the existing constraint-based planning tool, Tumato, incorporates the notion of uncertainty and state rules. Robustness against uncertainty is realized by extending actions with alternative, less desirable yet foreseeable outcomes. Further, state rules offer a powerful means to express safety constraints based on state conditions, alleviating the need for manual specification of all potential situations leading to unsafe states. This new approach significantly reduces the number of required safety rules, making the specification less susceptible to errors. The connection with STPA is briefly discussed. Tumato translates the declarative specification, including the safety rules, into a set of Constraint Satisfaction Problems (CSPs). The solution to these CSPs yields an execution policy that inherently adheres to all specified rules. A notable advantage is the early detection of unrealizable specifications, and ensuring that the obtained policy is sound and complete by construction.

Appendix A The full specification

In this appendix, we present the extended format of specification supported by Tumato, using the AMR example.

```
BEGIN STATE VECTOR
    state S_Location can be corridor, charger, workstation_1, workstation_2
    state S_Battery can be low, ok
    state S_Load can be loaded, free
    state S_Conveyor can be on, off
END STATE VECTOR

BEGIN RESOURCES
    resource MOTORS
    resource CONVEYOR
END RESOURCES

BEGIN ACTIONS
    action move_to_workstation_1
    duration: 10
    controlled resources: MOTORS
    preconditions: S_Location is corridor
    nominal effects: S_Location is workstation_1
    alternative effects: S_Location is corridor

    action move_to_workstation_2
    duration: 10
    controlled resources: MOTORS
    preconditions: S_Location is corridor
    nominal effects: S_Location is workstation_2
```

```
    action move_to_charger
    duration: 10
    controlled resources: MOTORS
    preconditions: S_Location is corridor
    nominal effects: S_Location is charger

    action move_to_corridor
    duration: 2
    controlled resources: MOTORS
    preconditions: NOT S_Location is corridor
    nominal effects: S_Location is corridor

    action receive_workpiece
    duration: 3
    controlled resources: MOTORS, CONVEYOR
    preconditions: S_Location is workstation_1, S_Load is free
    nominal effects: S_Conveyor is on, S_Load is loaded
    alternative effects: S_Conveyor is on, S_Load is free
    alternative effects: S_Conveyor is off, S_Load is free

    action deliver_workpiece
    duration: 3
    controlled resources: MOTORS, CONVEYOR
    preconditions: S_Location is workstation_2, S_Load is loaded
    nominal effects: S_Conveyor is on, S_Load is free
    alternative effects: S_Conveyor is on, S_Load is loaded

    action stop_conveyor
    duration: 1
    controlled resources: CONVEYOR
    preconditions: S_Conveyor is on
    nominal effects: S_Conveyor is off

    action charge
    duration: 50
    controlled resources: MOTORS
    preconditions: S_Location is charger
    nominal effects: S_Battery is ok
END ACTIONS

// Please note, comments start with "//".
BEGIN REACTION RULES
    // rule: IF (S_Location is corridor OR S_Location is charger) AND S_Conveyor is on
    //       THEN executing stop_conveyor
    // rule: IF (S_Location is corridor OR S_Location is charger) AND S_Conveyor is off
    //       THEN NOT executing receive_workpiece AND NOT executing deliver_workpiece
    //       AND NOT any future action that could turn the conveyor on
    // rule: IF S_Conveyor is on THEN NOT executing move_to_corridor AND NOT executing
    //       move_to_workstation_1 //AND NOT any future action that could move the AMR
    //       away from NOT(corridor OR charger).
    // And probably more rules
END REACTION RULES

BEGIN STATE RULES
    rule: IF S_Location is corridor OR S_Location is charger THEN S_Conveyor is off

    // Or even shorter, using an alias:
    // rule: IF NOT is_docked THEN S_Conveyor is off
END STATE RULES
```

```
BEGIN ASSUMPTIONS
    // No assumptions necessary in this example
END ASSUMPTIONS

BEGIN ALIASES
    define is_docked as S_Location is workstation_1 OR S_Location is workstation_2
END ALIASES

BEGIN GOALS
    goal type: priority

    when S_Battery is low
        then goal: S_Battery is ok
    when S_Load is loaded
        then goal: S_Load is free
    when S_Load is free
        then goal: S_Load is loaded
END GOALS

BEGIN CONFIG
    max_plan_length: 5
END CONFIG
```

Appendix B The full policy

In this appendix, we use the AMR example to illustrate a complete policy as generated by Tumato.

```
  "S_Location": "corridor",                  "S_Location": "corridor",
  "S_Battery": "ok",                         "S_Battery": "low",
  "S_Load": "free",                          "S_Load": "loaded",
  "S_Conveyor": "off",                       "S_Conveyor": "on",
  "Actions": ["move_to_workstation_1"]       "Actions": ["stop_conveyor"]

  "S_Location": "charger",                   "S_Location": "workstation_1",
  "S_Battery": "low",                        "S_Battery": "low",
  "S_Load": "free",                          "S_Load": "free",
  "S_Conveyor": "on",                        "S_Conveyor": "off",
  "Actions": ["stop_conveyor"]               "Actions": ["move_to_corridor"]

  "S_Location": "workstation_2",             "S_Location": "workstation_2",
  "S_Battery": "ok",                         "S_Battery": "low",
  "S_Load": "loaded",                        "S_Load": "loaded",
  "S_Conveyor": "off",                       "S_Conveyor": "off",
  "Actions": ["deliver_workpiece"]           "Actions": ["move_to_corridor"]

  "S_Location": "charger",                   "S_Location": "workstation_1",
  "S_Battery": "ok",                         "S_Battery": "low",
  "S_Load": "loaded",                        "S_Load": "loaded",
  "S_Conveyor": "on",                        "S_Conveyor": "off",
  "Actions": ["stop_conveyor"]               "Actions": ["move_to_corridor"]

  "S_Location": "charger",                   "S_Location": "charger",
  "S_Battery": "low",                        "S_Battery": "ok",
  "S_Load": "loaded",                        "S_Load": "free",
  "S_Conveyor": "on",                        "S_Conveyor": "off",
  "Actions": ["stop_conveyor"]               "Actions": ["move_to_corridor"]
```

```
"S_Location": "corridor",
"S_Battery": "low",
"S_Load": "free",
"S_Conveyor": "off",
"Actions": ["move_to_charger"]

"S_Location": "corridor",
"S_Battery": "low",
"S_Load": "loaded",
"S_Conveyor": "off",
"Actions": ["move_to_charger"]

"S_Location": "workstation_2",
"S_Battery": "ok",
"S_Load": "loaded",
"S_Conveyor": "on",
"Actions": ["deliver_workpiece"]

"S_Location": "charger",
"S_Battery": "low",
"S_Load": "loaded",
"S_Conveyor": "off",
"Actions": ["charge"]

"S_Location": "workstation_1",
"S_Battery": "ok",
"S_Load": "free",
"S_Conveyor": "on",
"Actions": ["receive_workpiece"]

"S_Location": "corridor",
"S_Battery": "ok",
"S_Load": "free",
"S_Conveyor": "on",
"Actions": ["stop_conveyor"]

"S_Location": "workstation_2",
"S_Battery": "low",
"S_Load": "loaded",
"S_Conveyor": "on",
"Actions": ["stop_conveyor"]

"S_Location": "workstation_2",
"S_Battery": "ok",
"S_Load": "free",
"S_Conveyor": "off",
"Actions": ["move_to_corridor"]

"S_Location": "charger",
"S_Battery": "ok",
"S_Load": "free",
"S_Conveyor": "on",
"Actions": ["stop_conveyor"]

"S_Location": "corridor",
"S_Battery": "ok",
"S_Load": "loaded",
"S_Conveyor": "on",
"Actions": ["stop_conveyor"]

"S_Location": "workstation_1",
"S_Battery": "ok",
"S_Load": "loaded",
"S_Conveyor": "on",
"Actions": ["stop_conveyor"]

"S_Location": "charger",
"S_Battery": "low",
"S_Load": "free",
"S_Conveyor": "off",
"Actions": ["charge"]

"S_Location": "charger",
"S_Battery": "ok",
"S_Load": "loaded",
"S_Conveyor": "off",
"Actions": ["move_to_corridor"]

"S_Location": "workstation_2",
"S_Battery": "ok",
"S_Load": "free",
"S_Conveyor": "on",
"Actions": ["stop_conveyor"]

"S_Location": "workstation_2",
"S_Battery": "low",
"S_Load": "free",
"S_Conveyor": "on",
"Actions": ["stop_conveyor"]

"S_Location": "workstation_1",
"S_Battery": "low",
"S_Load": "free",
"S_Conveyor": "on",
"Actions": ["stop_conveyor"]

"S_Location": "corridor",
"S_Battery": "ok",
"S_Load": "loaded",
"S_Conveyor": "off",
"Actions": ["move_to_workstation_2"]

"S_Location": "workstation_1",
"S_Battery": "ok",
"S_Load": "free",
"S_Conveyor": "off",
"Actions": ["receive_workpiece"]

"S_Location": "workstation_1",
"S_Battery": "low",
"S_Load": "loaded",
"S_Conveyor": "on",
"Actions": ["stop_conveyor"]

"S_Location": "workstation_2",
"S_Battery": "low",
"S_Load": "free",
"S_Conveyor": "off",
"Actions": ["move_to_corridor"]
```

```
"S_Location": "workstation_1",
"S_Battery": "ok",
"S_Load": "loaded",
"S_Conveyor": "off",
"Actions": ["move_to_corridor"]
```

```
"S_Location": "corridor",
"S_Battery": "low",
"S_Load": "free",
"S_Conveyor": "on",
"Actions": ["stop_conveyor"]
```

Acknowledgements This research is partially funded by the Research Fund KU Leuven.

Author Contributions J.V. and T.H. jointly contributed to the research presented in this manuscript. J.V. primarily took the lead in obtaining, organizing and writing the content. T.H., in the role of the promotor, provided valuable insights, critical comments, and guidance throughout the research process. Additionally, T.H. contributed to the manuscript by reviewing and proofreading. Both authors participated in discussions, revisions, and finalizing the manuscript and approved the final version for submission.

Data Availability No datasets were generated or analysed during the current study.

Declarations

Competing interests The authors declare no competing interests.

References

1. Buysse, L., Whiteley, S., Vankeirsbilck, J., et al.: Case study analysis of stpa on an industrial cooperative robot and an autonomous mobile robot. In: The Future of Safe Systems-Proceedings of the Safety-Critical Systems Symposium, SCSC on Amazon, pp. 83–104 (2023). https://lirias.kuleuven.be/retrieve/708014
2. Camacho, A., Triantafillou, E., Muise, C.J., et al.: Non-deterministic planning with temporally extended goals: completing the story for finite and infinite ltl. In: Proceedings of the Workshop on Knowledge-based Techniques for Problem Solving and Reasoning co-located with 25th International Joint Conference on Artificial Intelligence (IJCAI 2016), New York City, USA, July 10, 2016 (2016). https://ceur-ws.org/Vol-1648/paper10.pdf
3. Canal, G., Cashmore, M., Krivić, S., et al.: Probabilistic planning for robotics with rosplan. In: Towards Autonomous Robotic Systems: 20th Annual Conference, TAROS 2019, London, UK, July 3–5, 2019, Proceedings, Part I 20, pp. 236–250. Springer (2019). https://doi.org/10.1007/978-3-030-23807-0_20
4. Cashmore, M., Fox, M., Long, D., et al.: Rosplan: planning in the robot operating system. In: Proceedings of the International Conference on Automated Planning and Scheduling, pp. 333–341 (2015). https://doi.org/10.1609/icaps.v25i1.13699
5. Dechter, R., Meiri, I., Pearl, J.: Temporal constraint networks. Artif. Intell. **49**(1–3), 61–95 (1991). https://doi.org/10.1016/0004-3702(91)90006-6
6. Dinh, H.T., Cruz Torres, M.H., Holvoet, T.: Sound and complete reactive uav behavior using constraint programming (2017). https://lirias.kuleuven.be/retrieve/470086
7. Doherty, P., Kvarnström, J.: Talplanner: a temporal logic-based planner. AI Mag. **22**(3), 95–95 (2001). https://doi.org/10.1609/aimag.v22i3.1581
8. Doherty, P., Gustafsson, J., Karlsson, L., et al.: Tal: temporal action logics language specification and tutorial. Electron. Trans. Artif. Intell. (1998). http://www.ep.liu.se/ej/etai/1998/009/
9. Emerson, E.A.: Temporal and modal logic. In: Formal Models and Semantics, pp. 995–1072. Elsevier (1990). https://doi.org/10.1016/B978-0-444-88074-1.50021-4
10. Figat, M., Zieliński, C., Hexel, R.: Fsm based specification of robot control system activities. In: 2017 11th International Workshop on Robot Motion and Control (RoMoCo), pp. 193–198. IEEE (2017). https://doi.org/10.1109/RoMoCo.2017.8003912
11. Geffner, T., Geffner, H.: Compact policies for fully observable non-deterministic planning as sat. In: Proceedings of the International Conference on Automated Planning and Scheduling, pp. 88–96 (2018). https://doi.org/10.1609/icaps.v28i1.13880
12. Gerevini, A.E., Haslum, P., Long, D., et al.: Deterministic planning in the fifth international planning competition: Pddl3 and experimental evaluation of the planners. Artif. Intell. **173**(5–6), 619–668 (2009). https://doi.org/10.1016/j.artint.2008.10.012

13. Ghallab, M., Nau, D., Traverso, P.: Automated Planning and Acting, 1st edn. Cambridge University Press, USA (2016). https://doi.org/10.1017/CBO9781139583923
14. Karpas, E., Magazzeni, D.: Automated planning for robotics. Annu. Rev. Control Robot. Auton. Syst. **3**, 417–439 (2020). https://doi.org/10.1146/annurev-control-082619-100135
15. Leveson, N.G.: Engineering a safer world: systems thinking applied to safety. The MIT Press (2016)
16. Leveson, N.G., Thomas, J.P.: STPA Handbook. MIT (2018). http://psas.scripts.mit.edu/home/getfile.php?name=STPA_handbook.pdf
17. Maniatopoulos, S., Schillinger, P., Pong, V., et al.: Reactive high-level behavior synthesis for an atlas humanoid robot. In: 2016 IEEE International Conference on Robotics and Automation (ICRA), pp. 4192–4199. IEEE (2016). https://doi.org/10.1109/ICRA.2016.7487613
18. Nguyen, H., Ciocarlie, M., Hsiao, K., et al.: Ros commander (rosco): behavior creation for home robots. In: 2013 IEEE International Conference on Robotics and Automation, pp. 467–474. IEEE (2013). https://doi.org/10.1109/ICRA.2013.6630616
19. Prud'homme, C., Fages, J.G.: Choco-solver: a java library for constraint programming. J. Open Source Softw. **7**(78), 4708 (2022). https://doi.org/10.21105/joss.04708, https://choco-solver.org
20. Puterman, M.L.: Markov decision processes: discrete stochastic dynamic programming. John Wiley & Sons (2014). https://doi.org/10.1002/9780470316887
21. Tsang, E.: Foundations of constraint satisfaction. Academic Press Limited (1993)
22. Vermaelen, J., Holvoet, T.: Safe and robust robot behavior planning via constraint programming. Electron. Proc. Theor. Comput. Sci. **391**, 26–41 (2023). https://doi.org/10.4204/EPTCS.391.5
23. Vermaelen, J., Dinh, H.T., Holvoet, T.: A survey on probabilistic planning and temporal scheduling with safety guarantees. In: ICAPS Workshop on Planning and Robotics (2020). https://icaps20subpages.icaps-conference.org/wp-content/uploads/2020/10/12-PlanRob_2020_paper_12.pdf
24. Wongpiromsarn, T., Topcu, U., Murray, R.M.: Synthesis of control protocols for autonomous systems. Unmanned Syst. **1**(01), 21–39 (2013). https://doi.org/10.1142/S2301385013500027

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.